

CS3106L Operating Systems Laboratory

Lab 7: Deterministic Proportional-Share Scheduling in LUIT

Stride scheduling, fair admission, wakeup compensation, and SMP-safe selection

1. Starting point

Use the supplied LUIT_Lab07 repository directly. It is a complete LUIT checkpoint: kernel sources, user library, HAL, filesystem builder, top-level Makefile, completed dependencies from earlier labs, public programs, checking tools, and this handout are already included. Do not copy files from Lab 6, another LUIT tree, or xv6.

The completed Lab 6 mailbox implementation is already part of this checkpoint. The only Lab 7 implementation file that may be changed is:

```
kernel/proc.c
```

Only the six marked regions S1--S6 and the optional LAB7-HELPERS region are editable. Keep every marker intact.

Before editing anything:

```
python3 tools/lab07_check.py
make clean
make
make grade
```

The release checker must pass before work begins. A build failure in the untouched checkpoint indicates an environment/toolchain problem, not a scheduling task.

2. What is being implemented

The baseline LUIT scheduler scans the process table and runs a RUNNABLE process. Replace only its scheduling *policy* with deterministic proportional-share stride scheduling. The context-switch *mechanism* is supplied and must remain unchanged.

Each process has the supplied fields:

```
int sched_tickets;
uint64 sched_stride;
uint64 sched_pass;
uint64 sched_dispatches;
int sched_last_hart;
uint64 sched_migrations;
```

The constants are supplied in kernel/schedinfo.h:

```
STRIDE_DEFAULT_TICKETS = 100
STRIDE_MIN_TICKETS = 1
STRIDE_MAX_TICKETS = 1000
STRIDE_BIG = 10000000
```

For a process with T tickets:

$$\text{stride} = \left\lfloor \frac{\text{STRIDE_BIG}}{T} \right\rfloor.$$

More tickets therefore produce a smaller stride. At every dispatch, select the RUNNABLE process with the smallest `sched_pass`; before it runs, add its stride to its pass.

3. Exact Lab 7 scheduling contract

The following rules define correctness. Do not substitute a different stride variant.

3.1. Tickets and stride

Every fresh process slot starts with 100 tickets. `settickets(n)` accepts only $1 \leq n \leq 1000$. An invalid request returns `-1` and changes nothing. A valid request updates tickets and stride but **does not reset pass**. Resetting pass would erase already received CPU service and create a fairness exploit.

A child created by `fork()` inherits the parent's ticket count. Its Lab 7 statistics begin from zero.

3.2. Pass and the global floor

A sleeping or newly admitted process must not return with an old very small pass and monopolize the CPU while it "catches up." The checkpoint therefore supplies a global `stride_floor`, plus:

```
stride_floor_snapshot();
stride_floor_publish(value);
```

The floor is the pass value of the most recently selected global minimum, measured *before* that selected process is charged its stride.

Use these rules:

- A fresh slot starts at the current floor.
- On `fork()`, the child pass is at least the current floor and must not be less than the parent's current pass.
- When `wakeup()` changes a process from SLEEPING to RUNNABLE, clamp its pass upward to at least the current floor.
- If `kill()` wakes a sleeping process, apply the same clamp.

This is the Lab 7 answer to the "new arrivals need a sensible initial pass" and stale-sleeper problems discussed with stride scheduling.

3.3. Selection order

For each scheduling decision, compare only RUNNABLE processes. The key order is:

1. smaller `sched_pass`;
2. if pass is equal, prefer a process whose `sched_last_hart` equals the current hart;
3. if still tied, smaller PID.

The second rule is a small affinity-aware tie break. It must never override a lower pass value.

3.4. Statistics

Exactly once for every actual dispatch:

- increment `sched_dispatches`;
- if `sched_last_hart` is valid and differs from the current hart, increment `sched_migrations`;
- set `sched_last_hart` to the current hart;
- charge `sched_pass += sched_stride`.

Do not count a process merely because it was inspected during a scan.

4. SMP locking contract

This lab is not a single-hart simulator. Several LUIT scheduler loops may execute concurrently.

The supplied `stride_lock` serializes the global selection/charge operation. The intended lock discipline is:

1. acquire `stride_lock`;
2. inspect one process at a time while holding that process's `p->lock`;
3. never hold two process locks simultaneously while scanning;
4. identify the best candidate, then reacquire that candidate's lock;
5. while both locks are held, confirm it is still `RUNNABLE`, charge its stride, update Lab 7 statistics, and mark it `RUNNING`;
6. release `stride_lock`;
7. return the selected process with only `p->lock` still held.

Never hold `stride_lock` across `swtch()`. Doing so can stop every other hart from making a scheduling decision while the selected process runs.

The supplied `scheduler()` body performs the address-space switch, Lab 3 publication, `swtch()`, kernel page-table restoration, and final process-lock release. Do not move those operations into an editable region.

5. Task S1 - fresh process state

Complete S1 inside `allocproc()`.

Initialize all Lab 7 fields. The process must start with default tickets and the corresponding stride, zero dispatch/migration counters, no previous hart, and a pass that does not start behind the current system floor.

A reused process-table slot must not retain scheduler history from its previous owner. The supplied `freeproc()` already clears the fields when a process is reclaimed.

6. Task S2 - fork inheritance without a head start

Complete S2 in `fork()`.

The child inherits the parent's tickets and therefore the corresponding stride. Do not copy the parent's dispatch or migration counters. Use a pass value that is at least both the parent's current pass and the current floor.

Why this matters: assigning every child pass zero lets a process repeatedly fork new children that jump ahead of existing runnable work.

7. Task S3 - dynamic ticket changes

Complete `set_tickets()` in S3.

Requirements:

- reject values outside 1-1000;
- protect the current process's scheduler fields with its process lock;
- recompute stride from the supplied constant;
- preserve the accumulated pass;
- return 0 on success and -1 on rejection.

The syscall boundary and user stub are already supplied. Do not edit `kernel/schedlab.c`, `kernel/syscall.tbl`, or `user/ulib.h`.

8. Task S4 - global stride selection

Complete `stride_pick_next()` in S4. This is the main task.

The helper must return either:

- 0, with no locks held, when no process is `RUNNABLE`; or
- a pointer to one process that has already been changed to `RUNNING`, with `p->lock` held and `stride_lock` released.

Implement the exact key order given earlier. Publish the selected pre-charge pass as the new floor, then charge the selected process and update statistics exactly once.

Do not return a pointer obtained from an unlocked scan and then blindly run it. Another hart may have selected it. The final candidate state must be checked while its process lock is held and while the global selection lock still excludes another stride decision.

9. Tasks S5 and S6 - wakeup compensation

S5 modifies `wakeup()`; S6 modifies the sleeping-process branch of `kill()`.

Preserve the original process-state semantics. The additional Lab 7 rule is only this: immediately before a sleeping process becomes `RUNNABLE`, clamp its pass to the current floor if its saved pass is smaller.

Do not change a process's ticket count on sleep or wakeup. Do not reset its pass to zero. Do not reward an I/O-bound process with an arbitrary new priority; this lab remains a proportional-share scheduler.

10. Supplied observability API

Two Lab 7 system calls are already wired:

```
int settickets(int tickets);
int schedinfo(struct schedinfo *out);
```

`schedinfo()` returns a snapshot of the calling process:

```
struct schedinfo {
    int pid;
    int tickets;
    uint64 stride;
    uint64 pass;
    uint64 dispatches;
    int last_hart;
    uint64 migrations;
};
```

These calls exist to make the scheduling policy observable and testable. They are not an invitation to modify the ABI.

11. Public verification

Use the complete public suite:

```
python3 tools/lab07_check.py
make clean
make
make grade LAB=7
```

The public Lab 7 programs are protected infrastructure:

Program	What it exercises
<code>stride_api</code>	default policy, input validation, stride calculation, fork inheritance
<code>stride_share</code>	100:50:250 deterministic proportional share on one hart
<code>stride_wakeup</code>	stale-pass sleeper compensation after a long blocking interval
<code>stride_smp</code>	progress and ticket weighting under four-hart contention; scheduler statistics

Expected final markers are:

```
STRIDE_API: PASS
STRIDE_SHARE: PASS
STRIDE_WAKEUP: PASS
STRIDE_SMP: PASS
LAB7 PUBLIC: PASS
```

The public tests are not exhaustive. In particular, do not assume that one favorable SMP schedule proves correctness.

12. Failure patterns to diagnose

Symptom	Likely reason
All workers receive similar shares	baseline scan still chooses processes; pass is not the primary key
A new child dominates immediately	child pass incorrectly initialized to zero or below the floor
A process dominates after long blocking	wakeup pass was not clamped
System hangs on multiple harts	global selection lock held across <code>swtch()</code> , or incorrect process-lock ownership
Same process runs on two harts	candidate selected from an unlocked stale snapshot without protected final claim
Ticket change gives sudden huge advantage	<code>settickets()</code> reset pass
Migration counter exceeds dispatches	migration counted during scans rather than actual dispatches
Baseline tests regress	context-switch mechanism or non-Lab-7 source was changed

For GDB, the most useful breakpoints are `stride_pick_next`, `yield`, `sleep`, `wakeup`, and `swtch`. Inspect `sched_tickets`, `sched_stride`, `sched_pass`, and `sched_last_hart` while holding the relevant process lock.

13. Submission archive

After the complete public suite passes:

```
python3 tools/package_lab07.py YOUR_NINE_DIGIT_ROLL
```

This creates:

```
YOUR_NINE_DIGIT_ROLL_Lab07.zip
`-- YOUR_NINE_DIGIT_ROLL_Lab07/
    |-- kernel/
    |-- proc.c
```

The packager creates its temporary archive inside the repository, so it does not depend on `/tmp` being on the same filesystem. Do not use a manual ZIP workaround.

14. Final checklist

Before packaging, verify all of the following:

- only the six TODO regions/helper region in `kernel/proc.c` were edited;
- every TODO marker is still present;
- no file was copied from another LUIT/xv6 tree;
- `stride_lock` is never held across `swtch()`;
- no scan holds two process locks at once;
- invalid ticket requests leave scheduler state unchanged;
- `fork` does not create a pass-zero head start;
- `wakeup` and `kill` do not create a stale-pass burst;
- all four public markers appear and the shell returns after each test;
- `python3 tools/lab07_check.py` passes immediately before packaging.